

Transformers & Attention Mechanism (Part 1)

Introduction

Before Transformers, different neural network architectures were designed for different types of data.

Artificial Neural Networks (ANN)

ANN learns patterns from **fixed-size data** where the order of inputs does not matter.

Convolutional Neural Networks (CNN)

CNN is designed for **images**. It learns **spatial patterns** such as edges, textures, shapes, and objects, but it cannot understand sequential information.

Recurrent Neural Networks (RNN)

RNN is designed for **Sequential Data**, especially text.

It processes one word after another while remembering previous information through hidden states.

Simple RNN suffered from the **Vanishing Gradient Problem**, so improved architectures were introduced:

- LSTM (Long Short-Term Memory)
- GRU (Gated Recurrent Unit)

These solved long-term memory problems much better.

Different Types of Sequence Models

Many-to-One

Example:

Sentiment Analysis

Lots of words



One prediction

Example:

Positive

Negative

Neutral

Many-to-Many

Many inputs



Many outputs

Sequence-to-Sequence

Input sequence



Output sequence

Example:

English



French

Question



Answer

Summary Generation



Summary

Encoder–Decoder Architecture

Definition

An **Encoder–Decoder model** first understands the input sequence and then generates a new output sequence based on that understanding.

Workflow:

Input Sequence



Encoder



Context Vector



Decoder



Output Sequence / Summary

Why Encoder–Decoder Models?

Traditional Machine Learning models and early Deep Learning models mainly performed:

Input



Output

Examples:

- Machine Learning Algorithms

- ANN
- CNN

For sequential problems we introduced:

Sequence Models



RNN



LSTM



GRU

But when we needed to generate an entirely new sequence (such as translation or question answering), we required a **Sequence-to-Sequence Model**, which gave rise to **Encoder–Decoder Architecture**.

Examples:

Question



Answer

English Language



French Language

Text



Summary

This architecture became one of the foundations of modern **Generative AI**.

Encoder

Definition

The **Encoder** converts an input sequence into a meaningful internal representation (called the **Context Vector**) that captures the information of the entire sentence.

In simple words:

The encoder **understands** the input but does not generate any output.

Example

Training Phase

Input:

I love Sheriyans

Output:

Mujha Sheriyans Pasand Hai

3 Input Words



4 Output Words

The Encoder can be implemented using:

- Simple RNN
- LSTM
- GRU

The last hidden state (H_3) is called the **Context Vector**.

Characteristics of Encoder

- Uses RNN, LSTM, or GRU
 - Reads the input sequence from left to right
 - Produces a Context Vector
 - Does **not** generate output
 - Its only job is to understand the sentence
-

Context Vector

Definition

A **Context Vector** is the final hidden representation produced by the encoder that contains the summarized meaning of the entire input sentence.

It is passed to the decoder to generate the output sequence.

Decoder

Definition

The **Decoder** generates the output sequence one token (word) at a time using the Context Vector.

Unlike the Encoder, the Decoder is responsible for prediction.

Decoder Working

The Decoder behaves differently during:

- Training Phase
- Prediction (Inference) Phase

Training Phase (Teacher Forcing)

Definition

Teacher Forcing is a training technique in which the decoder is provided with the correct previous word instead of using its own prediction.

At every step, the decoder already knows the correct output.

This helps the model learn faster and more accurately.

Prediction Phase

During prediction, the decoder does **not** know the correct output.

Instead, it uses its own previous prediction to generate the next word.

Start Token

The Decoder always begins with a **Start Token**.

The Start Token, together with the Context Vector, becomes the first input to the decoder.

Softmax

The Decoder uses the **Softmax Activation Function** because every output word is treated as a multi-class classification problem.

Example:

Softmax Output

[0.1, 0.3, 0.4, 0.5]

Each value represents the probability of a possible next word.

Normally, the word with the highest probability is selected.

Why Teacher Forcing?

Suppose the decoder predicts the wrong word at an early stage.

If that incorrect prediction is used as the next input, future predictions also become incorrect.

To solve this problem, during training we use **Teacher Forcing**, where the correct previous word is provided instead of the predicted one.

Example:

Start Token

•

Context Vector



"Mujha"

instead of using the model's own incorrect prediction.

Characteristics of Decoder

- Probability-based model
- Softmax is applied during both training and prediction
- Depends on the previous word as input

- Cannot handle Out-of-Vocabulary (OOV) words
 - Generates one word at a time
-

Problems with Encoder–Decoder Models

Although Encoder–Decoder architecture was a major improvement over traditional RNNs, it still suffered from several important limitations.

1. Information Bottleneck

The entire sentence must be compressed into a **fixed-size Context Vector**.

Example:

If we have a very long sentence or a large log statement, the encoder has to summarize all information into a single fixed-length vector (for example, size = 4).

This creates a heavy burden on the encoder and important information may be lost.

2. No Word-to-Word Alignment

Different languages often require direct relationships between specific words.

The fixed Context Vector cannot preserve these alignments effectively.

3. Exposure Bias

Teacher Forcing trains the decoder using the correct previous word.

However, during prediction, the decoder must use its own predictions.

This mismatch between training and testing is called **Exposure Bias**.

4. Long Sentence Failure

As sentence length increases, the Context Vector becomes insufficient to represent the complete meaning.

Performance drops significantly for long sequences.

Attention Mechanism

Definition

The **Attention Mechanism** allows the decoder to focus on the most relevant parts of the input sentence instead of relying only on one fixed Context Vector.

Instead of using a single summary for the whole sentence, the decoder receives a **new Context Vector for every output word**.

This helps the model pay attention to the right information at the right time.

Example

Sentence:

How are you brother

Initial Context:

[0, 0, 0, 0]

Instead of using only one Context Vector,
the decoder receives a new context at every time
step.

Example:

T_1 in Decoder

$[S_0, Y_0]$



Decoder with Attention

$[S_0, Y_0] + C_1$

Here, C_1 is a new Context Vector generated
specifically for the current word.

Calculating Context Values

The Context Vector is calculated by combining all encoder hidden states using learned attention weights.

Formula:

$$C_i = \sum (\text{Attention Weight} \times \text{Hidden State})$$

The Attention Mechanism uses the **Universal Approximation capability of Artificial Neural Networks** to learn these weights.

During training, the network:

- Learns appropriate attention weights
- Adjusts model parameters
- Generates better Context Vectors

Through these repeated optimization steps, the Attention Mechanism learns where to focus.

Advantages of Attention Mechanism

- Allows the decoder to look at all encoder hidden states.

- Helps focus on distant but important words.
 - Improves training speed slightly.
 - Introduces word alignment between input and output.
 - Produces better Context Vectors than a single fixed representation.
-

Problems That Still Remained

Although Attention solved many problems, it still depended on RNNs.

Remaining limitations:

- RNN processing is sequential.
- Information must travel through time step by step.
- Parallel computation is not possible.
- Architecture remains complex.
- Training is still relatively slow.

These remaining limitations led to the development of **Transformers**, which completely removed RNNs and relied entirely on Attention Mechanisms.
